certificateSigning HTTP/TLS/JSON

Interface Design Description

Service ID: *"certificateSigning"*

Abstract
This document describes the HTTP/1.1 protocol with TLS payload security and JSON payload encoding variant of the `certificateSigning` service of the CertificateAuthority system.

Contents

1 Overview **3**
1.1 Interface registration 3

2 Service Operations **4**
2.1 operation sign 4

3 Data Models **5**
3.1 struct CertificateSigningRequest 5
3.2 struct CertificateSigningResult 5
3.3 Primitives 5

4 References **7**

5 Revision History **8**
5.1 Amendments 8
5.2 Quality Assurance 8

1 Overview

This document describes the `certificateSigning` service interface of the CertificateAuthority system, which provides signing of proof-of-possession certificate requests with the local-cloud CA, under the Arrowhead certificate profiles.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	HTTP	1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `certificateSigning` service interface

This document provides the Interface Design Description IDD to the *certificateSigning – Service Description* document [5]. For further details about how this service is meant to be used, please consult that document.

The rest of this document describes how to realize the `certificateSigning` service interface in detail. Both in terms of its operations (Section 2) and its information model (Section 3).

1.1 Interface registration

The service is registered in the ServiceRegistry with the interface template `generic_https`, policy `CERT_AUTH`, access address `certificate-authority`, port 8448 and base path `/certificate-authority`. The `operations` property of the interface lists the operations below with their method and path relative to the base path (path parameters are placeholders such as `{id}` that the consumer fills in):

Operation	Method	Path
`sign`	POST	`/sign`

2 Service Operations

The interfaces of the `certificateSigning` service, its operations, data models and implementation are provided below. In particular, each subsection first names the HTTP method and path used to call the operation, after which it names the abstract operation from the `certificateSigning` SD document, as well as input and output types.

Every request needs mutual TLS with a system or operator certificate of the cloud and `Authorization: Bearer IDENTITY-TOKEN//<token>`; the token is verified with Authentication and must belong to the system named by the certificate. Request and response bodies are `application/json` unless stated otherwise. Errors are answered with `{"error": "<text>"}` and the status codes `400 Bad Request` (invalid input), `401 Unauthorized` (no valid identity), `403 Forbidden` (caller not permitted), `404 Not Found`, `500 Internal Server Error`, `502 Bad Gateway` (a system it depends on failed) and `503 Service Unavailable` (overloaded).

2.1 POST /certificate-authority/sign

Operation: sign
Input: CertificateSigningRequest
Output: CertificateSigningResult

Signs a CSR for the requester's permitted profile and records the certificate.

A successful call returns `201 Created` with the output in the body.

```
POST /certificate-authority/sign HTTP/1.1
Host: certificate-authority:8448
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "encodedCSR": "-----BEGIN CERTIFICATE REQUEST-----\\n...",
  "profile": "sy",
  "addresses": [
    "sensor1"
  ],
  "validAfter": "2026-10-22T12:00:00Z",
  "validBefore": "2026-10-22T12:00:00Z"
}
```

Listing 1: An sign invocation.

```
{
  "id": 42,
  "certificateChain": [
    "<leaf Base64 DER>",
    "<CA Base64 DER>"
  ],
  "commonName": "Device1.TestCloud.Company.arrowhead.eu",
  "profile": "SYSTEM",
  "serialNumber": "4189327123456789",
  "validAfter": "2026-10-22T12:00:00Z",
  "validBefore": "2026-10-22T12:00:00Z"
}
```

Listing 2: An sign response.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is meant to denote a JSON Object that must contain certain fields, or names, with values conforming to explicitly named types. Fields that are not needed may be omitted or `null` in requests; empty fields are omitted in responses. As a complement to the primary types defined in this section, there is also a list of secondary types in Section 3.3, which are used to represent things like hashes, identifiers and texts.

3.1 struct CertificateSigningRequest

Object Field	Value Type	Description
"encodedCSR"	String	PEM PKCS\#10 request or Base64 DER (alias `csr`).
"profile"	String	`on`, `de`, `sy` or `op`; default the CSR dnQualifier, else `sy`. Must be allowed for the caller.
"addresses"	Array<String>	DNS names and IP addresses for the subject alternative name (at most 16); default those of the CSR.
"validAfter"	DateTime	Start of validity, ISO 8601; default now minus one minute.
"validBefore"	DateTime	End of validity; default start plus the profile maximum (24 h for `on`, 365 days otherwise), never beyond the CA certificate.

3.2 struct CertificateSigningResult

Object Field	Value Type	Description
"id"	Number	Record identifier in the certificate register.
"certificateChain"	Array<String>	Issued certificate and CA certificate, Base64 DER, leaf first.
"commonName"	String	Subject common name.
"profile"	String	Granted profile: `ON_BOARDING`, `DEVICE`, `SYSTEM` or `OPERATOR`.
"serialNumber"	String	Serial number, decimal.
"validAfter"	DateTime	Start of validity.
"validBefore"	DateTime	End of validity.

3.3 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available. Note that the official standard is defined in terms of parsing rules, while this list only concerns syntactic information. Furthermore, the Object and Array types are given optional generic type parameters, which are used in this document to signify when pair values or elements are expected to conform to certain types.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] R. Fielding, M. Nottingham, and J. Reschke, “Http/1.1,” IETF, RFC 9112, 2022.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “certificateSigning – Service Description (CertificateAuthority),” Eclipse Arrowhead, SD 5.2.2, 2026, certificateAuthority-v5.2.2-certificateSigning-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-10-30	5.2.2	Checked against the primary Java source tree (Eclipse-Arrowhead-java): services, operations, paths and data models unchanged; missing primitive definitions added.	Claude Opus 5.5 (draft)

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	